Assessment-9

Student Name | Register Number | BCSE308P – Course Project

1. Project Title

DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go

2. Abstract

The Domain Name System (DNS) translates human-readable domain names into IP addresses and is used by nearly every connection on the Internet. Because classic DNS is unauthenticated, a recursive resolver that accepts a forged reply can be made to cache false records and redirect all of its users, as the Kaminsky attack of 2008 showed [1]. Yet the component that decides which answers to trust is usually hidden inside large production resolvers or high-level libraries. This project presents DNSR, an iterative (recursive) DNS resolver written from scratch in the Go programming language without any DNS library. Every byte of the RFC 1035 wire format [2], including the header, questions, resource records, name compression and the EDNS(0) OPT record [3], is encoded and decoded by hand.

Starting from the 13 IANA root hints, DNSR sends non-recursive queries and walks the real DNS hierarchy (root, top-level domain, authoritative server). It follows referrals and glue records, resolves glueless delegations with nested lookups, chases CNAME aliases, tells NXDOMAIN apart from NODATA and falls back from UDP to TCP when a response is truncated. A TTL-aware answer cache and a delegation cache cut down repeated network round trips. The resolver defends against spoofing and cache poisoning with randomised transaction IDs and source ports, strict matching of reply ID and question, bailiwick checks on both referrals and glue, and a loop limit on compression pointers. It can be used as a command-line tool that prints a step-by-step trace, or as a recursive DNS server.

About 80% of the planned implementation is complete. All nine offline tests, which run against a simulated DNS hierarchy that includes malicious servers, pass. In live tests DNSR resolved 15 popular domains with the same response codes as Google Public DNS (median 571 ms with an

empty cache; under 1 ms from the cache), and 34.7 million random, mutated and coverage-guided inputs caused no failure in the parser. A code review identified the remaining hardening work, which forms the last 20% of the implementation.

Keywords: Domain Name System, iterative resolution, recursive resolver, cache poisoning, bailiwick, EDNS(0), Go, network security.

3. System Architecture, Design Methodology

3.1 System Architecture

DNSR follows a layered architecture (Figure 1). The interface layer offers two entry points: a command-line client (`main.go`) and a UDP recursive server (`server.go`). Both pass requests to the resolution engine (`resolver.go`), which holds the iterative algorithm, the security checks and the caches. The engine talks to the Internet through a transport layer that sends UDP queries with an EDNS(0) buffer of 1232 bytes [4] and switches to TCP when the TC bit is set [5]. All messages are built and parsed by the wire codec (`wire.go`), which implements the RFC 1035 binary format by hand. Each layer depends only on the one below it.

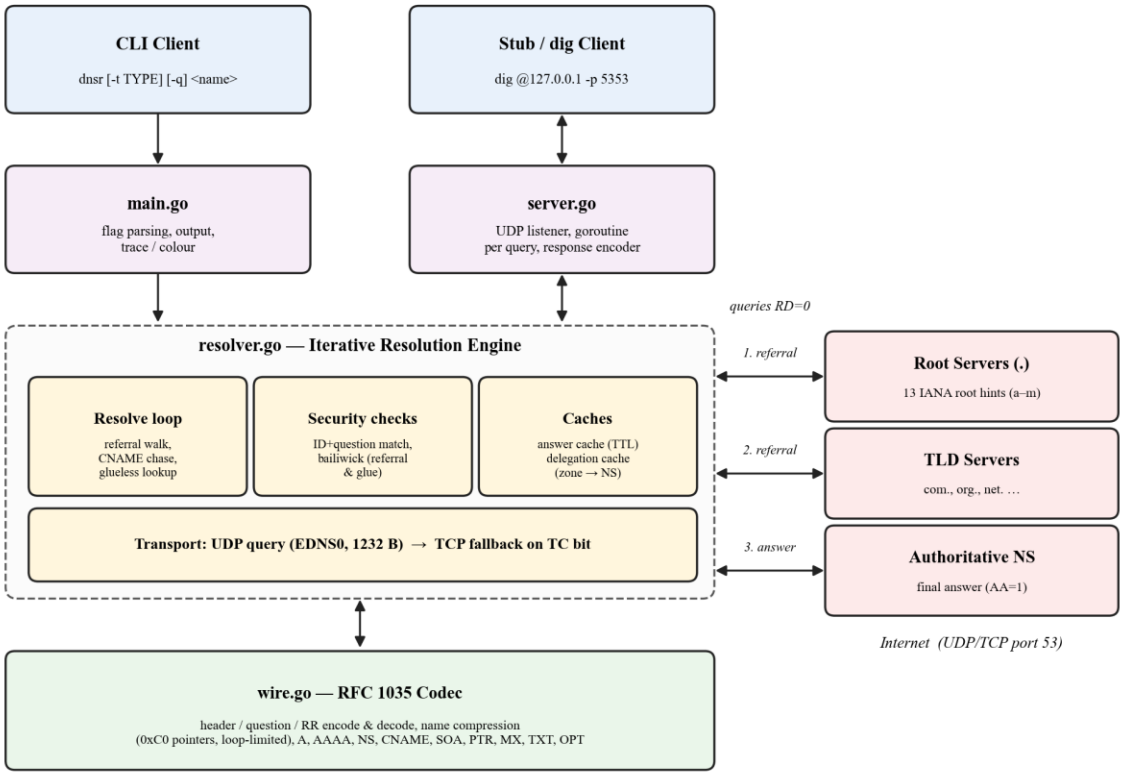


Figure 1: System architecture of DNSR

Table 1: Module description

Module (lines)	Responsibility
wire.go (320)	Header, Question, RR and Message types; encodes names and queries (RD = 0 with an EDNS(0) OPT record); decodes compressed names with a 64-jump loop limit; parses A, AAAA, NS, CNAME, PTR, MX, SOA and TXT data.
resolver.go (427)	Root hints, iterative resolution loop, answer cache with TTL count-down, delegation cache, referral and glue processing with bailiwick checks, glueless lookup, CNAME restart, random server selection, UDP exchange with anti-spoofing checks and TCP fallback.
server.go (133)	Listens on UDP, handles each query in its own goroutine, builds responses (QR = 1, RA = 1) and re-encodes names inside RDATA so that no stale compression pointers are sent.
main.go (89)	Parses CLI flags (-t type, -q quiet, -nocolor), selects CLI or server mode and prints the trace and the answer.
resolver_test.go (292)	Fake DNS hierarchy on 127.0.0.10 to 127.0.0.15 (root, com, org, example.com, dnshost.org, glueless.com) for deterministic offline tests.

3.2 Resolution Workflow

Figure 2 shows the control flow for a single query. The resolver first checks the answer cache. On a miss it picks the deepest zone it already knows from the delegation cache, falling back to the root servers, and sends a non-recursive query to a randomly chosen server of that zone. A reply is accepted only if its transaction ID and question match the query. Depending on the response, the resolver returns an answer, restarts for a CNAME target, returns a negative result, or processes a referral. A referral is accepted only if the child zone lies strictly below the current zone and contains the queried name. Glue records are accepted only when they fall inside the zone of the server that sent them; if no valid glue is left, the resolver resolves a name server's address with a nested lookup. The new delegation is cached and the loop continues one level closer to the target. Hop (40), depth (8), server (4 per step) and time (2 s per exchange) limits guarantee termination.

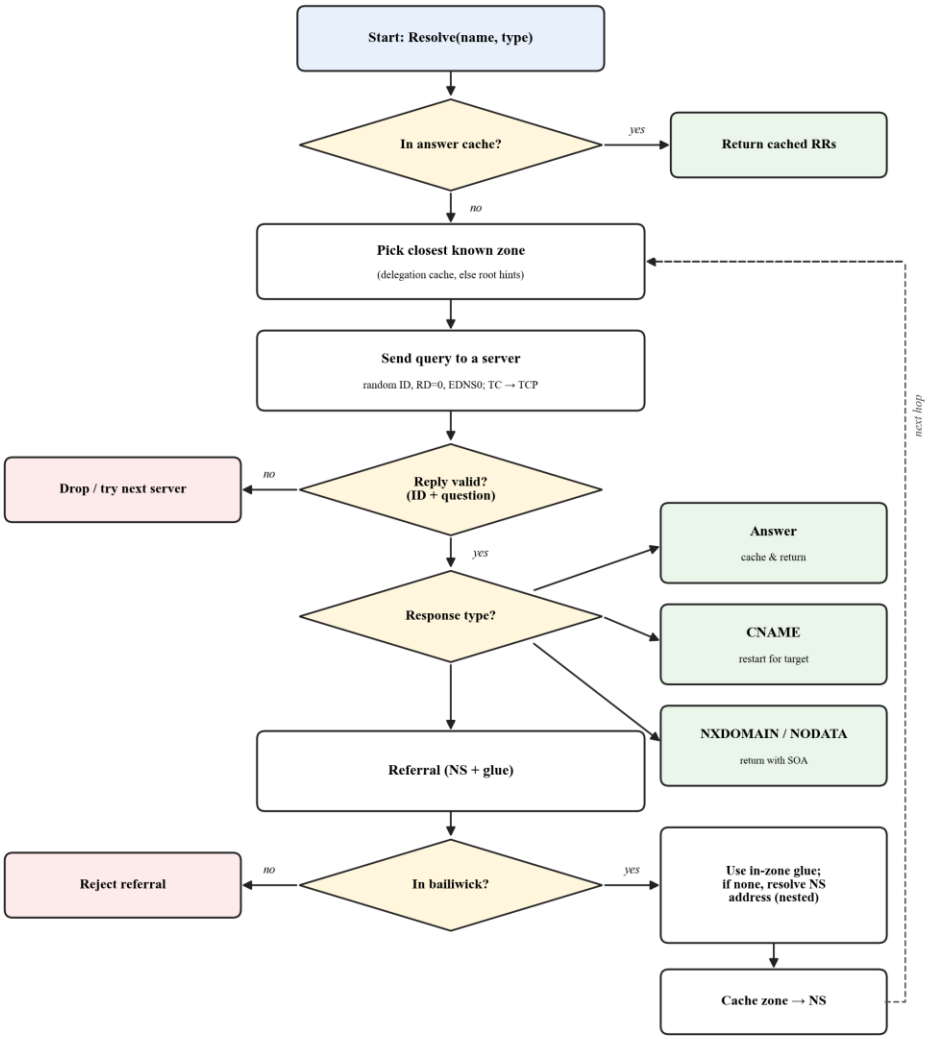


Figure 2: Iterative resolution flowchart

Figure 3 shows the messages of a real cold-cache resolution of `www.github.com` captured with DNSR. After two referrals the GitHub name server answers with a CNAME; DNSR restarts for `github.com` at the cached `github.com.` zone and obtains the address with one more query.

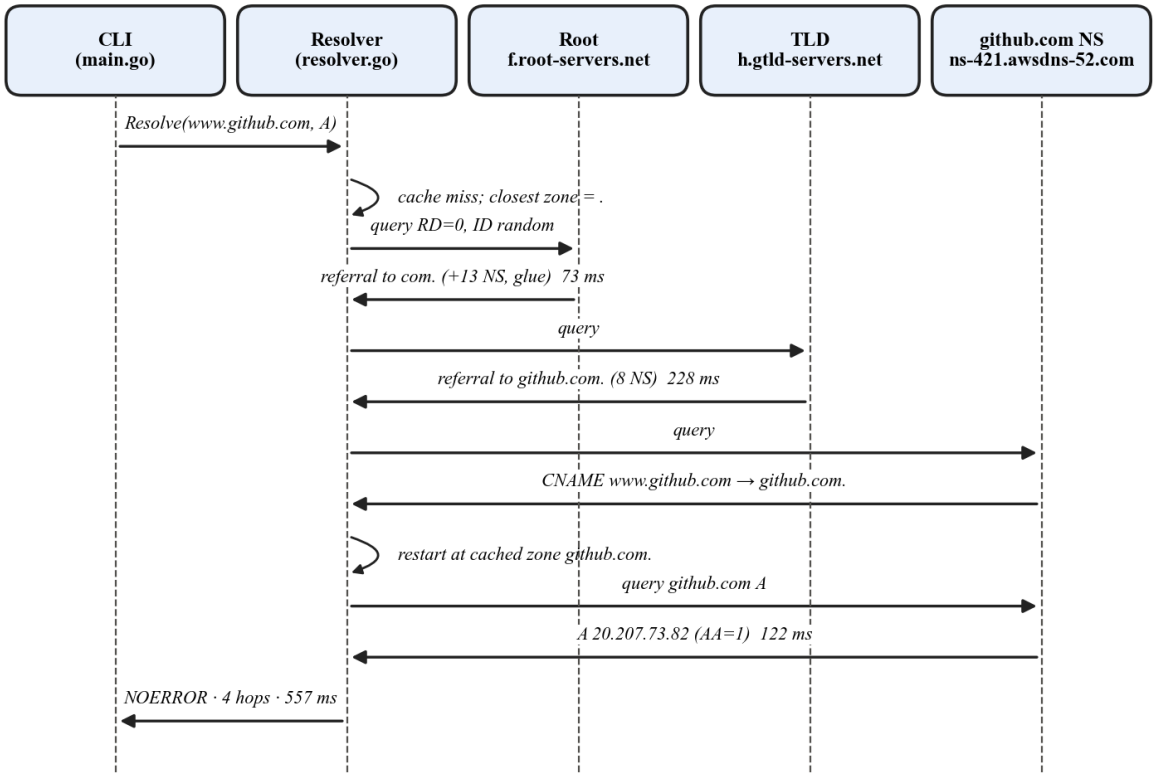


Figure 3: Message sequence for a cold-cache lookup of www.github.com

3.3 Security Design

The design defends against two adversaries: an *off-path spoofer* who cannot see queries but can send forged packets, and a *malicious authoritative server* that tries to affect names outside its own zone. An on-path attacker can only be stopped by DNSSEC [6], which is outside the scope of this version. Table 2 maps each threat to its defence.

Table 2: Threats and defences

Threat	Defence in DNSR
Blind spoofing of replies [1]	Random 16-bit ID; fresh connected UDP socket per query (random source port); reply accepted only if ID, QR, question name and type match [7]
Forged referral or glue (Kaminsky-style poisoning)	Referral must move strictly down towards the queried name; glue must belong to a referred NS and lie in the sending server's zone [8], [9]
Unrelated records slipped into a reply	Only answer records whose owner is the queried name are used and cached; CNAME targets are re-resolved, never taken from the same packet
Malformed packets	Bounds check before every read; at most 64 compression-pointer jumps; reserved label types rejected

Threat	Defence in DNSR
Loops and resource exhaustion	Limits of 40 hops, depth 8, 4 servers per step and 2 s per exchange; server listens on 127.0.0.1 by default

The bailiwick rules are illustrated in Figure 4 for replies from a `.com` server. The same rules are tested offline against a deliberately hostile `com.` server that offers poisoned glue and an out-of-zone referral.

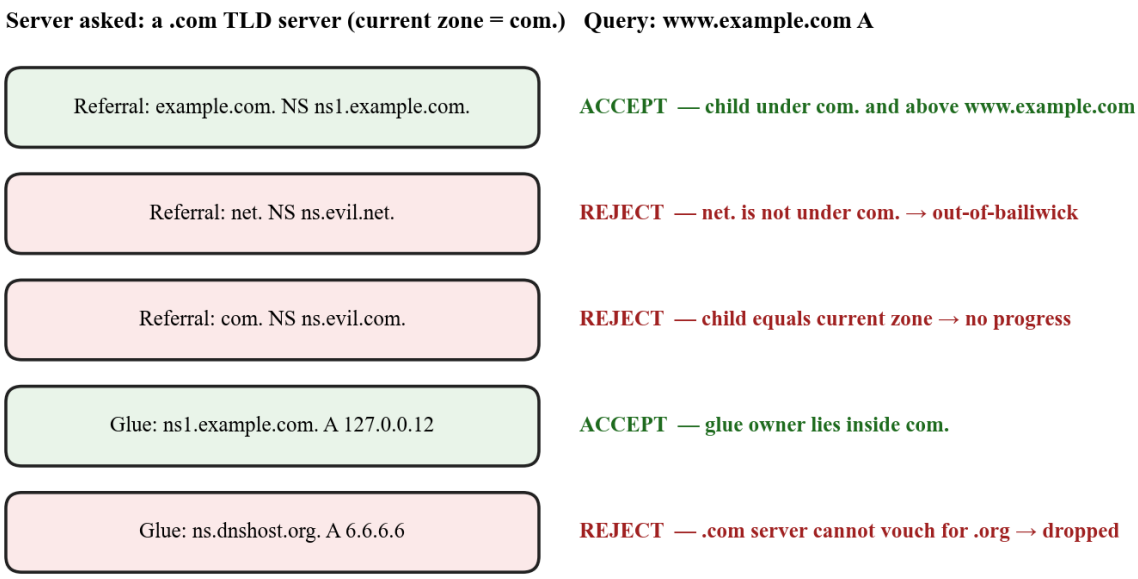


Figure 4: Bailiwick decisions for replies from a .com server

3.4 Design Methodology

The project follows an incremental, test-driven methodology in which each protocol feature is specified from the relevant RFC, implemented in isolation and verified before the next layer is built on top of it.

- ✓ **Requirement analysis and protocol study:** RFC 1034/1035 [10], [2], RFC 2181 (data ranking) [8], RFC 6891 (EDNS(0)) [3], RFC 7766 (TCP) [5] and RFC 5452 (forgery resilience) [7] were studied to set the functional and security requirements.
- ✓ **Wire-format codec:** message encoding and decoding were implemented by hand, with compression-pointer loop protection and bounds checks on every read.

- ✓ **Iterative resolution engine:** the referral-following loop from the root hints, then CNAME chasing, NXDOMAIN and NODATA detection [11], glueless-delegation handling and the loop limits.
- ✓ **Security hardening:** random IDs, strict reply matching and the two bailiwick checks, each covered by a test with a malicious server.
- ✓ **Caching and transport:** answer cache with remaining-TTL reporting, zone-to-NS delegation cache [12], randomised server selection with fail-over and UDP-to-TCP fallback.
- ✓ **Server mode:** the resolver exposed as a concurrent UDP DNS server.
- ✓ **Testing and validation:** offline simulated hierarchy, live measurements against 15 domains compared with Google Public DNS, fuzzing [13] and benchmarks.

Table 3: Tools and technologies

Component	Technology
Language	Go (standard library only: net, encoding/binary, sync, flag, math/rand) [14]
Transport	UDP and TCP to port 53; server mode on a configurable UDP address
Testing	Go testing package: fake multi-zone DNS hierarchy on 127.0.0.10–15, benchmarks, native fuzzing
Validation	dnspython 2.7.0 [15] as an independent client; Google Public DNS (8.8.8.8) as reference
Platform	Windows 11 (development and tests); builds for Linux and macOS as a single static binary

4. Implementation (80%)

4.1 Implementation Status

Table 4 shows the state of each component. All functional components of the resolver are implemented and tested, which together account for 80% of the planned work. The remaining 20% consists of the hardening and efficiency items found during the security review in Section 5.6; they will be completed for the final review. DNSSEC validation and IPv6 transport are planned extensions beyond the scope of this project.

Table 4: Implementation progress

Component	Status	Weight	Done
RFC 1035 wire codec (encode, decode, compression, EDNS(0))	Complete	15%	15%
Iterative resolution from root hints, referrals, glue	Complete	15%	15%
CNAME chasing, NXDOMAIN / NODATA, glueless delegations	Complete	10%	10%
Anti-spoofing: random ID and port, strict reply matching	Complete	8%	8%
Bailiwick checks on referrals and glue; resolution limits	Complete	8%	8%
Answer cache and delegation cache	Complete	6%	6%
UDP to TCP fallback	Complete	4%	4%
CLI with trace, quiet and colour options	Complete	4%	4%
Recursive server mode	Complete	4%	4%
Offline test hierarchy and 9 tests	Complete	6%	6%
Hardening: TTL cap, bounded caches, expiring delegations, glueless lookup limit	Pending	8%	0%
Server limits: worker pool, rate limit, access list; TCP question check; new default port	Pending	6%	0%
Negative caching (RFC 2308) and caching of CNAME records	Pending	6%	0%
Total		100%	80%

4.2 Wire Codec

The codec builds queries and parses replies by hand. The most security-sensitive function is `readName`, which decodes a possibly compressed domain name. It checks every offset against the message length, remembers where the name ends in the original stream, limits pointer jumps to 64 so that a pointer loop cannot hang the parser, and rejects the reserved label types.

```
// readName decodes a possibly-compressed name starting at off.
// Returns the name and the offset just past it in the *original* stream.
func readName(msg []byte, off int) (string, int, error) {
    var labels []string
    end := -1
    for jumps := 0; ; {
        if off >= len(msg) {
            return "", 0, errShort
        }
        l := int(msg[off])
        switch {
        case l == 0:
            if end < 0 {
                end = off + 1
            }
            return strings.Join(labels, ".") + ".", end, nil
```

```
        case l&0xC0 == 0xC0:
            if off+1 >= len(msg) {
                return "", 0, errShort
            }
            if end < 0 {
                end = off + 2
            }
            jumps++
            if jumps > 64 {
                return "", 0, errors.New("compression pointer loop")
            }
            off = int(binary.BigEndian.Uint16(msg[off:]) & 0x3FFF)
        case l&0xC0 != 0:
            return "", 0, fmt.Errorf("unsupported label type 0x%x", l)
        default:
            off++
            if off+l > len(msg) {
                return "", 0, errShort
            }
            labels = append(labels, string(msg[off:off+l]))
            off += l
        }
    }
}
```

`BuildQuery` writes a header with a random ID and all flags cleared (RD = 0, because DNSR performs the recursion itself), the question, and an EDNS(0) OPT record advertising a 1232-byte UDP buffer.

```
// BuildQuery makes a non-recursive query (RD=0) with an EDNS0 OPT record
// advertising a 1232-byte UDP buffer (the DNS Flag Day 2020 value).
func BuildQuery(id uint16, name string, qtype RRType) ([]byte, error) {
    b := make([]byte, 12, 512)
    binary.BigEndian.PutUint16(b[0:], id)
    binary.BigEndian.PutUint16(b[2:], 0)
    binary.BigEndian.PutUint16(b[4:], 1)
    binary.BigEndian.PutUint16(b[10:], 1)

    var err error
    if b, err = encodeName(b, name); err != nil {
        return nil, err
    }
    b = binary.BigEndian.AppendUint16(b, uint16(qtype))
    b = binary.BigEndian.AppendUint16(b, 1)

    b = append(b, 0)
    b = binary.BigEndian.AppendUint16(b, uint16(TypeOPT))
    b = binary.BigEndian.AppendUint16(b, 1232)
    b = binary.BigEndian.AppendUint32(b, 0)
    b = binary.BigEndian.AppendUint16(b, 0)
    return b, nil
}
```

4.3 Resolution Engine and Bailiwick Checks

The core loop in `resolve` queries a server of the current zone and classifies the reply. The part below handles a referral: it collects the NS records, rejects a referral that does not move strictly down towards the queried name, keeps only in-bailiwick glue and, if none is left, resolves a name server's address with a nested lookup before caching the new delegation.

```
var nsRRs []RR
var child string
for _, rr := range msg.Authority {
    if rr.Type == TypeNS {
        nsRRs = append(nsRRs, rr)
        child = strings.ToLower(rr.Name)
    }
}
if len(nsRRs) == 0 {
    return nil, fmt.Errorf("lame response from %s: no answer, no referral", ns.Name)
}

if !isSubdomain(name, child) || !isSubdomain(child, zone) || child == zone {
    return nil, fmt.Errorf("out-of-bailiwick referral from %s: %s is not under %s", ns.Name,
child, zone)
}

next := r.glueFor(nsRRs, msg.Additional, zone)
names := make([]string, 0, len(nsRRs))
for _, rr := range nsRRs {
    names = append(names, strings.TrimSuffix(rr.Data, "."))
}
r.tracef(depth, "%s %s → referral to %s %s (%s)", r.c(cCyan, "↓"), ns.Name, r.c(cBold, child),
    r.c(cDim, fmt.Sprintf("[%s]", summarize(names, 3))), rtt)

if len(next) == 0 {
    r.tracef(depth, "%s no glue for %s — resolving a nameserver's address first", r.c(cYellow,
"…"), child)
    for _, nsrr := range shuffled(nsRRs) {
        sub, err := r.resolve(fqdn(nsrr.Data), TypeA, depth+1, 0)
        if err != nil || sub.Rcode != RcodeNoError {
            continue
        }
        for _, a := range sub.Answers {
            if a.Type == TypeA {
                next = append(next, nameserver{fqdn(nsrr.Data), a.Data})
            }
        }
        if len(next) > 0 {
            break
        }
    }
    if len(next) == 0 {
        return nil, fmt.Errorf("could not find an address for any %s nameserver", child)
    }
}

r.mu.Lock()
r.zoneNS[child] = next
r.mu.Unlock()
zone, servers = child, next
```

```
// glueFor pairs NS names with A records from the Additional section.
// Glue is only trusted if it lies inside the zone of the server that sent it
// (askedZone) — a .com server may vouch for ns1.example.com, not for ns.evil.org.
func (r *Resolver) glueFor(nsRRs, additional []RR, askedZone string) []nameserver {
    want := map[string]bool{}
    for _, rr := range nsRRs {
        want[fqdn(rr.Data)] = true
    }
    var out []nameserver
    for _, rr := range additional {
        n := fqdn(rr.Name)
        if rr.Type == TypeA && want[n] && isSubdomain(n, askedZone) {
            out = append(out, nameserver{n, rr.Data})
        }
    }
    return out
}
```

4.4 Anti-Spoofing Exchange and TCP Fallback

`exchange` sends each query from a new connected UDP socket, so the operating system picks a random source port and discards datagrams from other addresses. It keeps reading until a reply passes every check or the two-second deadline expires; a forged or stray packet is ignored rather than treated as an error, so garbage cannot be used to block resolution. A truncated reply is retried over TCP.

```
// exchange sends one query over UDP, retrying over TCP if the answer is truncated.
func (r *Resolver) exchange(ip, name string, qtype RRType) (*Message, error) {
    id := uint16(rand.Uint32())
    q, err := BuildQuery(id, name, qtype)
    if err != nil {
        return nil, err
    }
    addr := net.JoinHostPort(ip, r.Port)

    conn, err := net.DialTimeout("udp", addr, r.Timeout)
    if err != nil {
        return nil, err
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(r.Timeout))
    if _, err := conn.Write(q); err != nil {
        return nil, err
    }
    buf := make([]byte, 4096)
    for {
        n, err := conn.Read(buf)
        if err != nil {
            return nil, err
        }
        m, err := ParseMessage(buf[:n])
        if err != nil {
```

```
            return nil, err
        }
        if m.Header.ID != id || !m.Header.QR || len(m.Questions) != 1 ||
            !strings.EqualFold(m.Questions[0].Name, name) || m.Questions[0].Type != qtype {
            continue
        }
        if m.Header.TC {
            r.tracef(0, "%s response truncated, retrying over TCP", r.c(cYellow, "⇄"))
            return exchangeTCP(addr, q, id, r.Timeout)
        }
        return m, nil
    }
}
```

4.5 Caching

The answer cache stores each RRset until its shortest TTL expires (Eqn. 1) and returns the remaining time as the TTL on every hit (Eqn. 2). The delegation cache maps zones to name servers; `closestKnownZone` walks up the queried name to the deepest cached zone, so a second lookup in the same domain skips the root and TLD servers.

$$t_{exp} = t_{store} + min_i\, TTL_i \tag{1}$$

$$TTL_{rem} = \lfloor\, t_{exp} - t_{now} \,\rfloor \tag{2}$$

```
func (r *Resolver) closestKnownZone(name string) (string, []nameserver) {
    r.mu.Lock()
    defer r.mu.Unlock()
    for z := name; ; {
        if ns, ok := r.zoneNS[z]; ok && len(ns) > 0 {
            return z, ns
        }
        if z == "." {
            return ".", r.Roots
        }
        if i := strings.IndexByte(z, '.'); i >= 0 && i+1 < len(z) {
            z = z[i+1:]
        } else {
            z = "."
        }
    }
}
```

4.6 Server Mode

`serve` reads datagrams in a loop and handles each one in a goroutine. Packets that cannot be parsed, are responses (QR = 1) or do not have exactly one question are dropped; failures are answered with SERVFAIL. Because names inside upstream RDATA may contain compression

pointers into the upstream packet, `encodeRData` re-encodes NS, CNAME, PTR, MX and SOA data before building the response.

```
func serve(addr string, r *Resolver) error {
    pc, err := net.ListenPacket("udp", addr)
    if err != nil {
        return err
    }
    log.Printf("dnsr listening on udp://%s — try: dig @%s -p %s example.com",
        pc.LocalAddr(), strings.Split(pc.LocalAddr().String(), ":")[0],
strings.Split(pc.LocalAddr().String(), ":")[1])

    buf := make([]byte, 1500)
    for {
        n, client, err := pc.ReadFrom(buf)
        if err != nil {
            return err
        }
        pkt := append([]byte(nil), buf[:n]...)
        go func() {
            m, err := ParseMessage(pkt)
            if err != nil || len(m.Questions) != 1 || m.Header.QR {
                return
            }
            q := m.Questions[0]
            res, err := r.Resolve(q.Name, q.Type)
            var out []byte
            if err != nil {
                log.Printf("%-30s %-5s SERVFAIL (%v)", q.Name, q.Type, err)
                out = buildResponse(m.Header.ID, q, m.Header.RD, RcodeServFail, nil, nil, nil)
            } else {
                log.Printf("%-30s %-5s %-8s %d answer(s), %d hops, %s", q.Name, q.Type,
                    rcodeString(res.Rcode), len(res.Answers), res.Hops, res.Elapsed.Round(1e6))
                out = buildResponse(m.Header.ID, q, m.Header.RD, res.Rcode, res.Answers,
res.Authority, nil)
            }
            pc.WriteTo(out, client)
        }()
    }
}
```

4.7 Test Suite

The tests start six small authoritative servers on loopback addresses 127.0.0.10 to 127.0.0.15 (Figure 5) and point the resolver at the fake root. The `com.` server is deliberately hostile: it delegates `trap.com.` with NS records for `net.` and attaches poisoned glue `ns.dnshost.org A 6.6.6.6` to the delegation of `glueless.com.`.

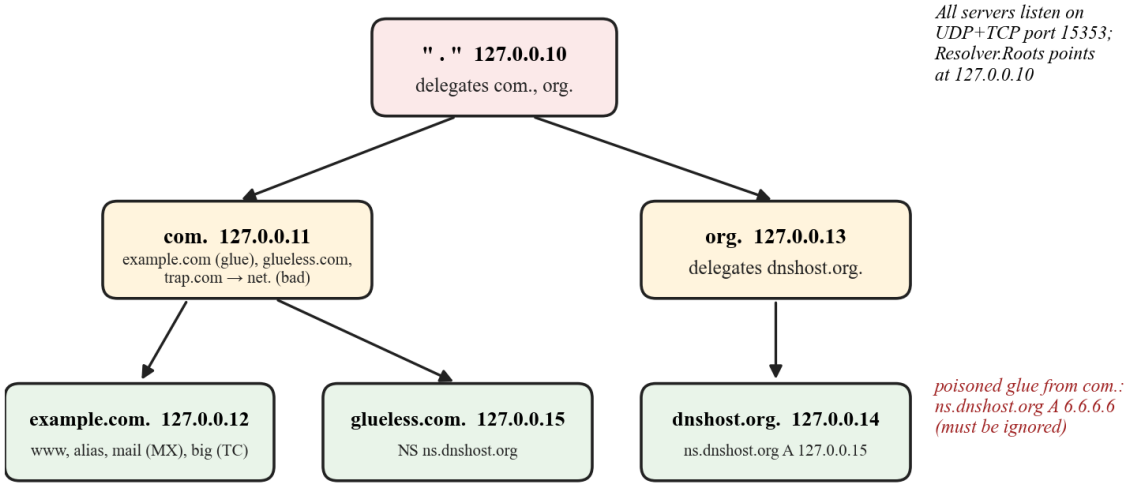


Figure 5: Offline test hierarchy on loopback addresses

```
func TestGluelessDelegationIgnoresPoisonedGlue(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("glueless.com", TypeA)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10.0.0.1" {
        t.Fatalf("%+v %v\n%s", res, err, trace)
    }
    if strings.Contains(trace.String(), "6.6.6.6") {
        t.Errorf("used out-of-bailiwick glue:\n%s", trace)
    }
    if !strings.Contains(trace.String(), "no glue") {
        t.Errorf("expected glueless path:\n%s", trace)
    }
}

func TestOutOfBailiwickReferralRejected(t *testing.T) {
    r, _ := newTestResolver(t)
    _, err := r.Resolve("www.trap.com", TypeA)
    if err == nil || !strings.Contains(err.Error(), "bailiwick") {
        t.Fatalf("expected bailiwick error, got %v", err)
    }
}
```

4.8 Working Implementation

The screenshots below show the implemented system resolving real names on the Internet. Figure 6 resolves `www.github.com` through a CNAME, Figure 7 handles a glueless delegation for `github.io`, and the following figures show TCP fallback, the caches, and server mode with an independent client.

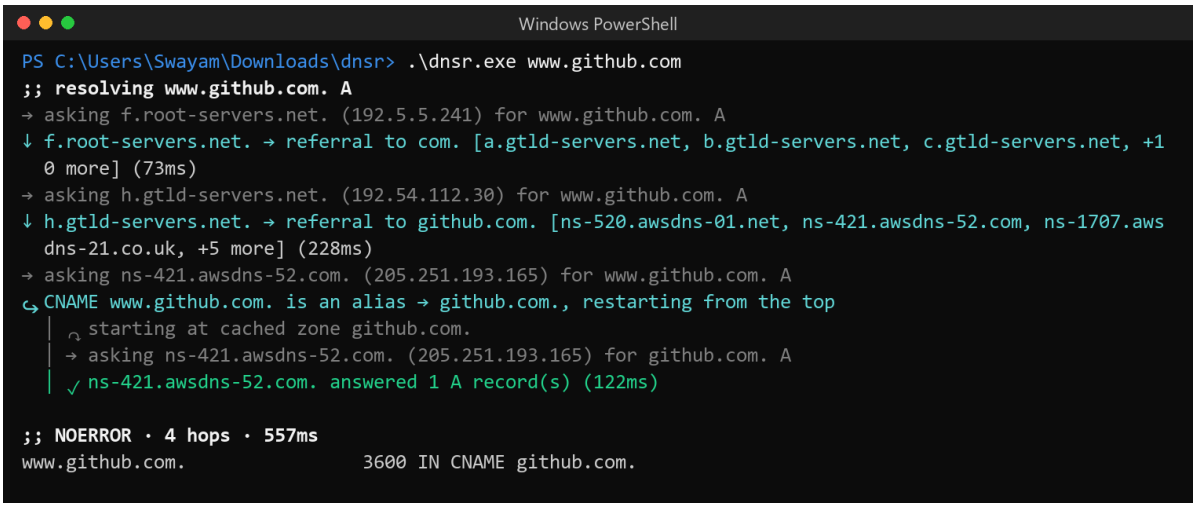


Figure 6: Iterative resolution of www.github.com with a CNAME

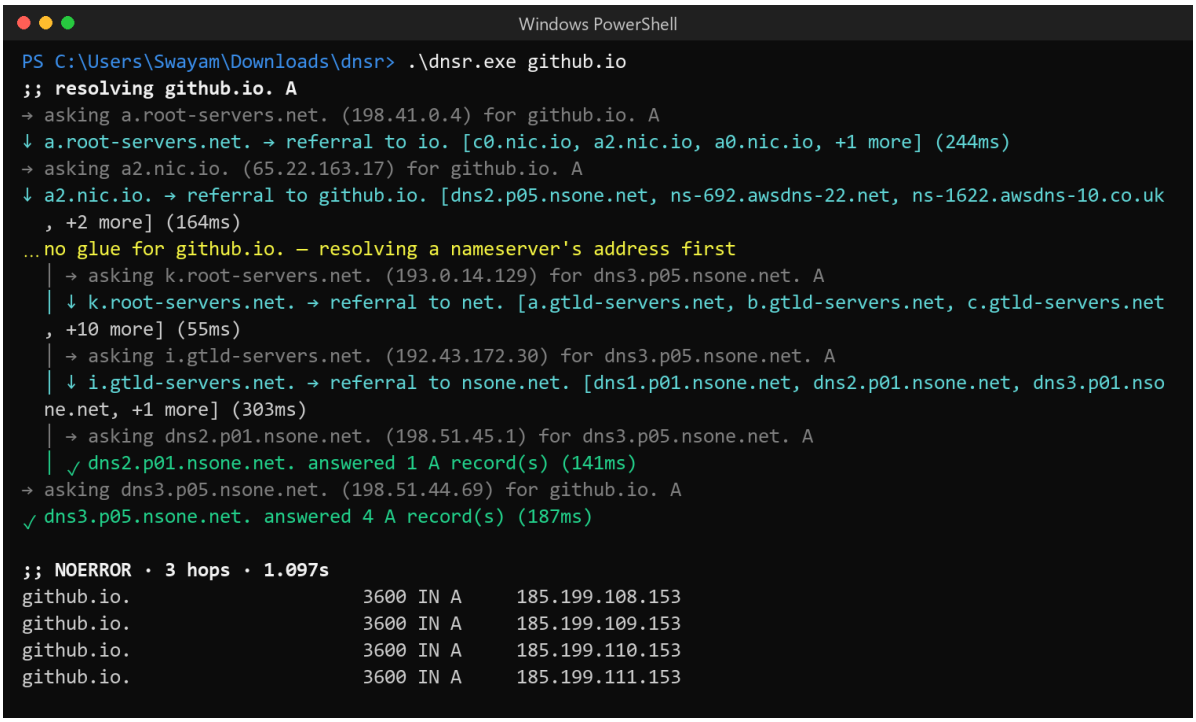


Figure 7: Glueless delegation for github.io

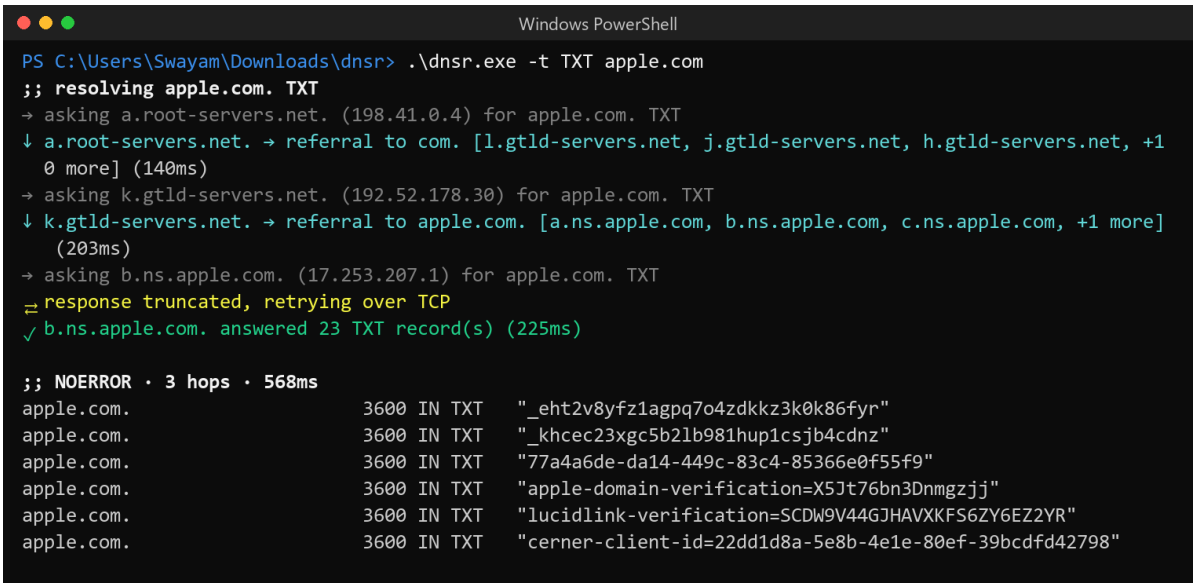


Figure 8: Truncated UDP reply retried over TCP (apple.com TXT)

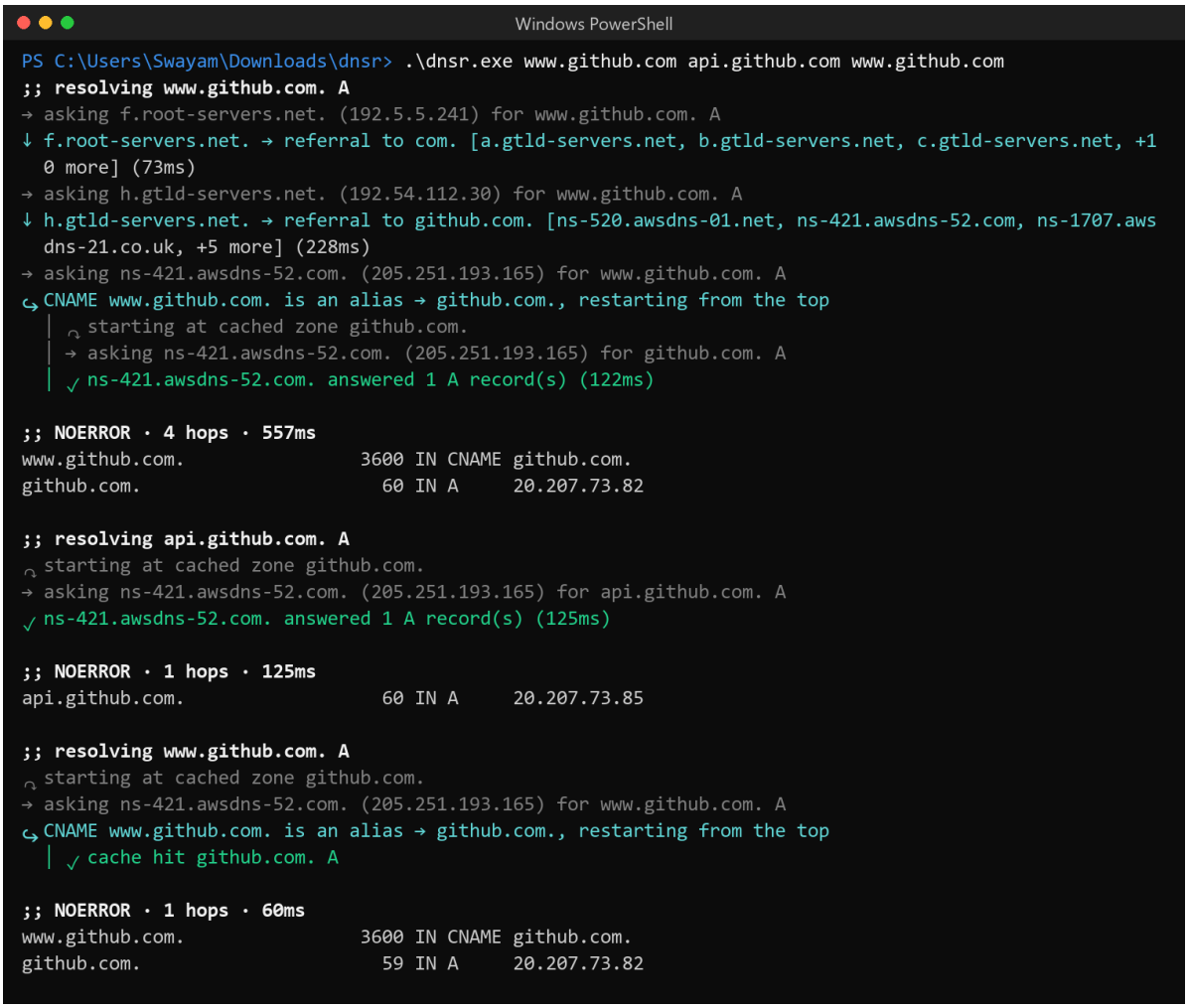


Figure 9: Delegation cache and answer cache in one run



Figure 10: DNSR running as a recursive server on 127.0.0.1:8053

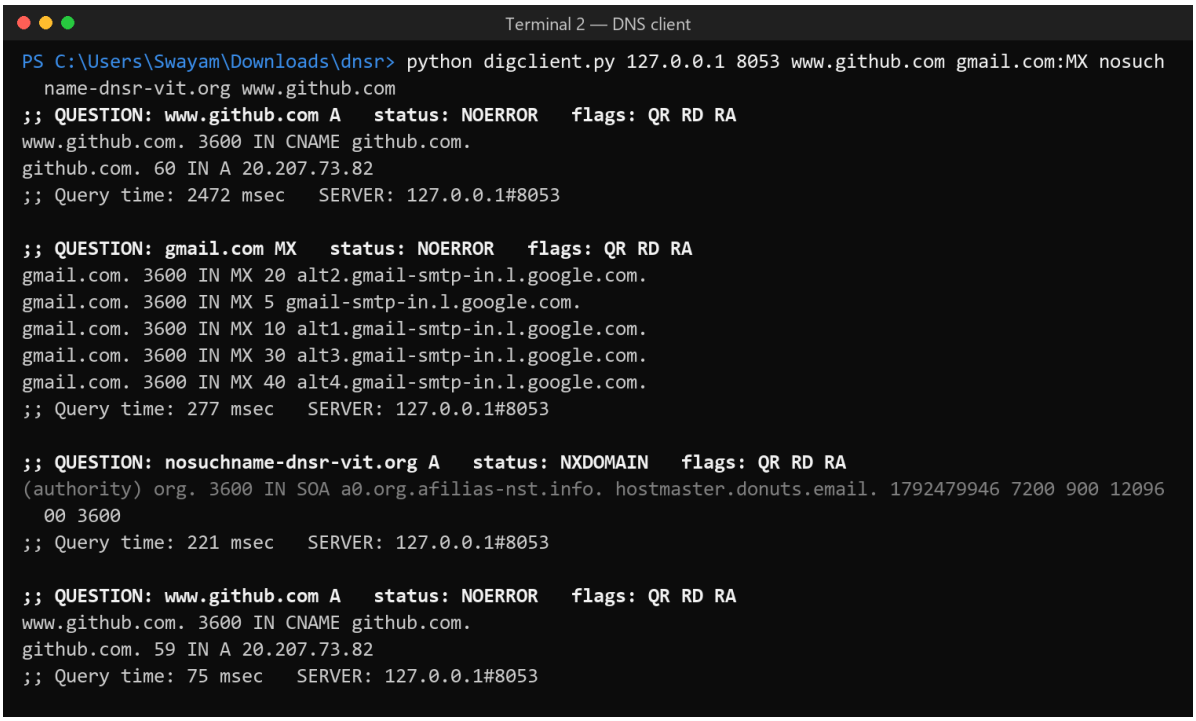


Figure 11: Independent DNS client (dnspython) querying the DNSR server

5. Findings and Result Analysis

5.1 Functional Tests

All nine offline tests pass (Figure 12); the whole suite runs in 1.74 s without network access.

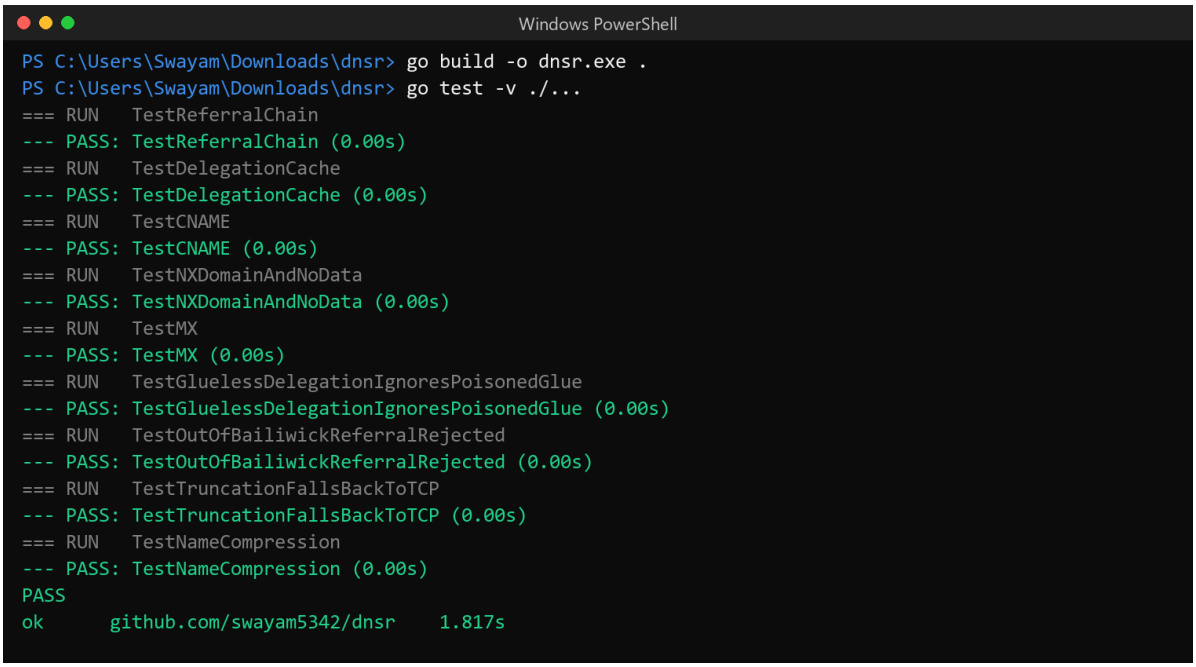


Figure 12: Build and offline test results

Table 5: Offline test results

Test	What it verifies	Result
TestReferralChain	Root to TLD to authoritative walk returns the correct A record	PASS
TestDelegationCache	A second lookup in the same zone skips root and TLD	PASS
TestCNAME	Alias followed; CNAME placed before the answer	PASS
TestNXDomainAndNoData	Non-existent names and missing types told apart	PASS
TestMX	MX preference and exchange parsed	PASS
TestGluelessDelegationIgnoresPoisonedGlue	Out-of-bailiwick glue discarded; NS address resolved independently	PASS
TestOutOfBailiwickReferralRejected	Redirection to an unrelated zone refused	PASS
TestTruncationFallsBackToTCP	TC = 1 reply retried over TCP	PASS
TestNameCompression	Compressed names decoded; pointer loop rejected	PASS

5.2 Live Correctness and Latency

Each of 15 popular domains was resolved three times by a fresh DNSR process (empty caches) on 20 October 2026, and the answers were compared with Google Public DNS (8.8.8.8). 43 of 45 runs succeeded; the two failures (one each for `vit.ac.in` and `go.dev`) were most likely server

timeouts, and the other runs for those domains succeeded. All 15 domains returned the same response code as 8.8.8.8, and 12 returned overlapping addresses. The three exceptions (`google.com`, `microsoft.com`, `gmail.com`) are served by content-delivery networks that give location-dependent answers, which is expected because DNSR asks the authoritative servers from the user's network.

Table 6: Live cold-cache results (three runs per domain)

Domain	OK	Hops	Mean (ms)	Range (ms)	Same as 8.8.8.8
google.com	3/3	3	414	408–418	Code yes; IP differs (CDN)
www.github.com *	3/3	4	1205	541–2525	Yes
www.wikipedia.org *	3/3	5	1517	773–2599	Yes
amazon.com	3/3	3	1087	424–2381	Yes
vit.ac.in	2/3	3	1975	1167–2783	Yes
microsoft.com	3/3	3	997	276–2243	Code yes; IP differs (CDN)
cloudflare.com	3/3	3	633	526–690	Yes
stackoverflow.com	3/3	3	506	419–602	Yes
example.com	3/3	3	1234	425–2799	Yes
gmail.com	3/3	3	749	571–1041	Code yes; IP differs (CDN)
netflix.com	3/3	3	733	519–1011	Yes
iana.org	3/3	3	452	422–473	Yes
python.org	3/3	3	328	238–444	Yes
go.dev	2/3	3	827	749–905	Yes
github.io	3/3	3	2763	744–4699	Yes

* answer reached through a CNAME.

The median cold-cache latency was 571 ms (mean 1011 ms, range 238 to 4699 ms), with 3.2 queries per name on average (Figure 13). The long tail comes from servers that do not answer within the two-second timeout, because servers are tried one at a time; RTT-based server selection would remove most of it.

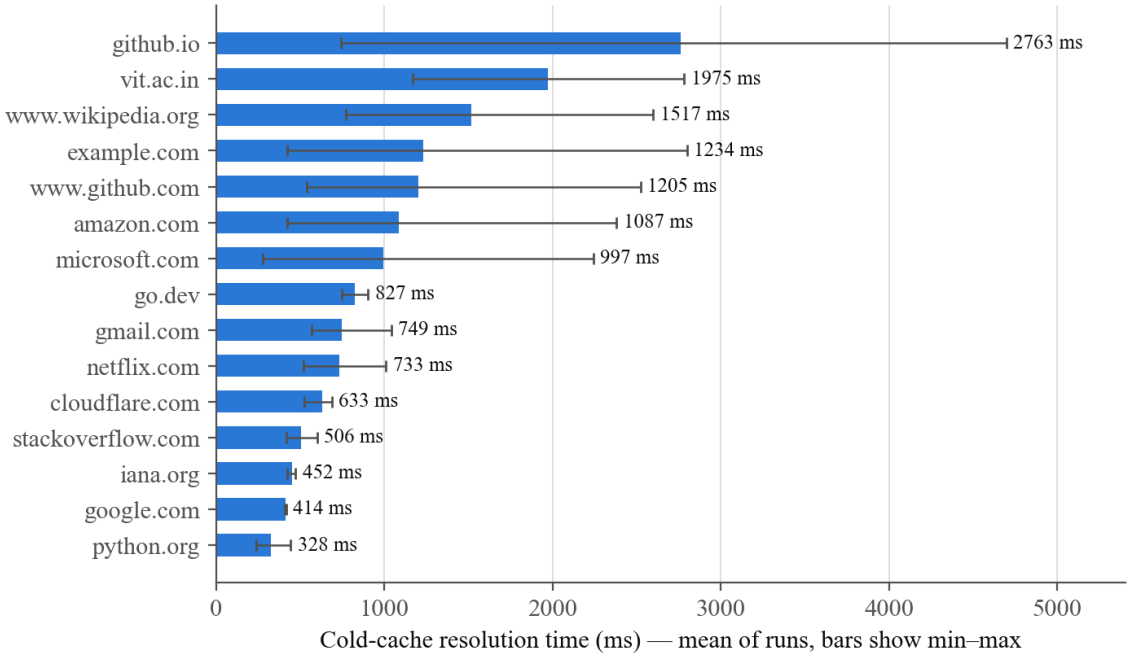


Figure 13: Cold-cache resolution time per domain

5.3 Effect of Caching

Five pairs of names were resolved in one process: a first name with empty caches, a sibling in the same domain (delegation cache), and the first name again (answer cache). The latency reduction is ρ = (L_cold − L_cached) / L_cold × 100% (Eqn. 3).

$$\rho = \frac{L_{cold} - L_{cached}}{L_{cold}} \times 100\% \quad (3)$$

Table 7: Latency with empty, delegation and answer caches (ms, hops in brackets)

First name / sibling	Cold	Sibling	ρ	Repeat	ρ
www.github.com / api.github.com	684 (4)	94 (1)	86%	32 (1)	95%
google.com / mail.google.com	583 (3)	89 (1)	85%	0 (0)	100%
www.wikipedia.org / en.wikipedia.org	1023 (5)	311 (1)	70%	91 (1)	91%
vit.ac.in / www.vit.ac.in	757 (3)	49 (1)	94%	0 (0)	100%
amazon.com / www.amazon.com	358 (3)	589 (4)	-65%	0 (0)	100%

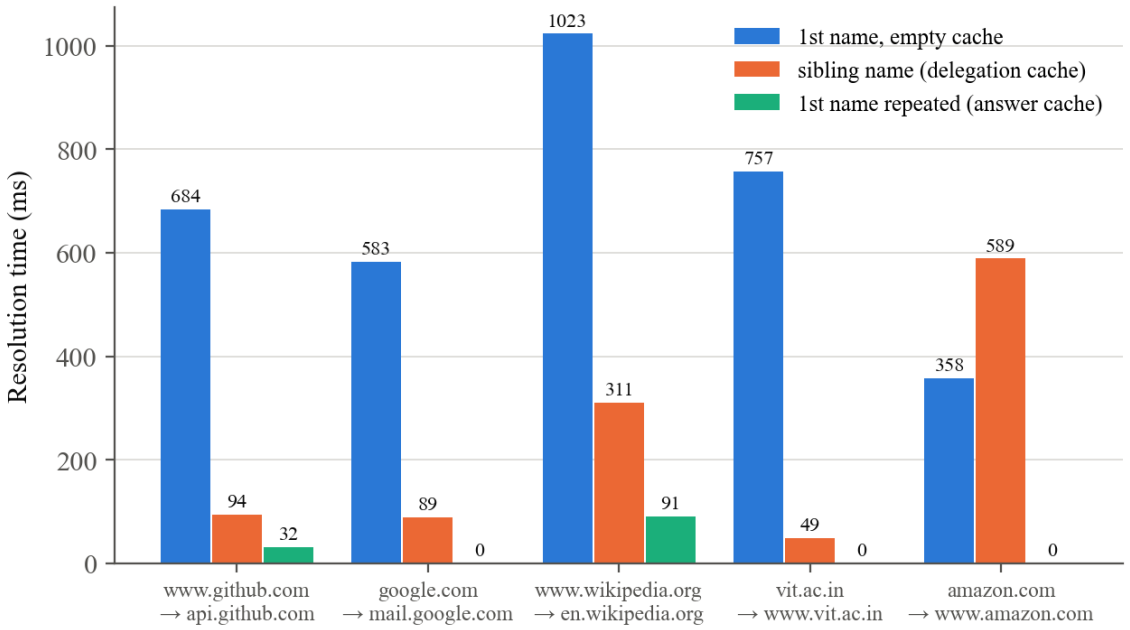


Figure 14: Effect of the delegation cache and the answer cache

The delegation cache cut sibling lookups by 67 to 94% in four of five cases, and plain repeated names were answered from the answer cache in under a millisecond. Two limitations were found: `www.amazon.com` is a CNAME into another provider's zone and needs a new walk, and a repeated alias (for example `www.github.com`) still costs one query because CNAME records are not cached under the alias name.

5.4 Server Mode

Started with `dnsr serve -addr 127.0.0.1:8053`, DNSR answered every query from dnspython with a valid message (flags QR, RD, RA). Repeated queries for cached names took 0.8 to 8.8 ms instead of 315 to 2,575 ms (Figure 15). A repeated NXDOMAIN query took 126 ms against 143 ms because negative answers are not yet cached. The default port 5353 could not be used on Windows, where it is taken by the multicast DNS service.

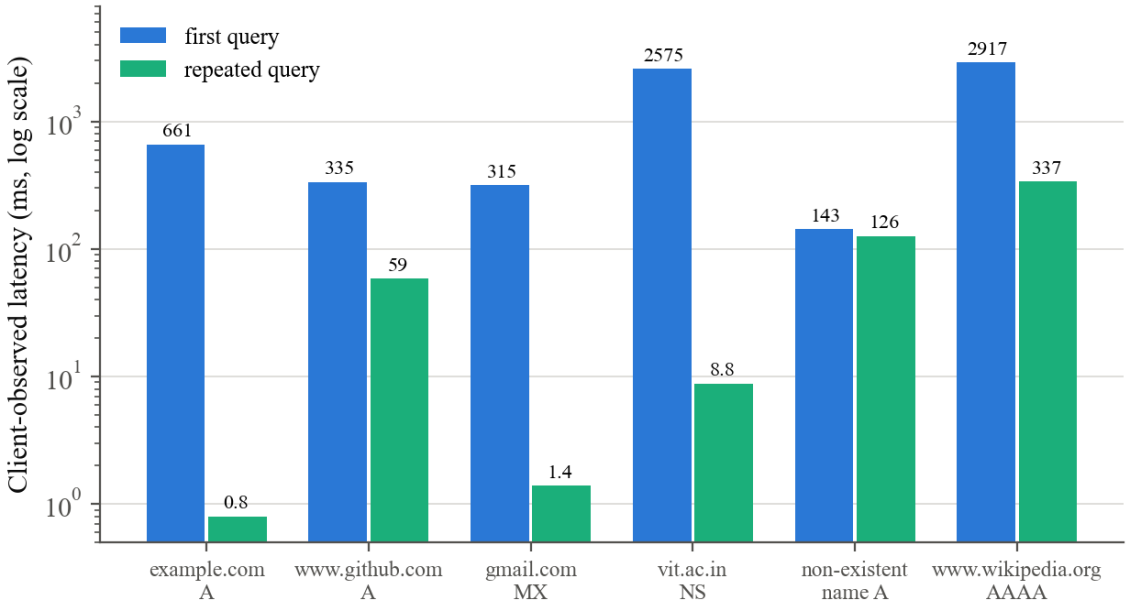


Figure 15: Server mode: latency of first and repeated queries

5.5 Security and Robustness

With a random ID only, an off-path attacker who delivers k forged replies in time succeeds with probability $P = 1 - (1 - 1/N)^k$ with $N = 2^{16}$; 45,000 packets give an even chance. The fresh source port per query multiplies N by the 16,384-port Windows ephemeral range [16], so the same chance needs about 7.4×10^8 packets (Figure 16). Answer filtering and the bailiwick checks limit what even a successful forgery could inject.

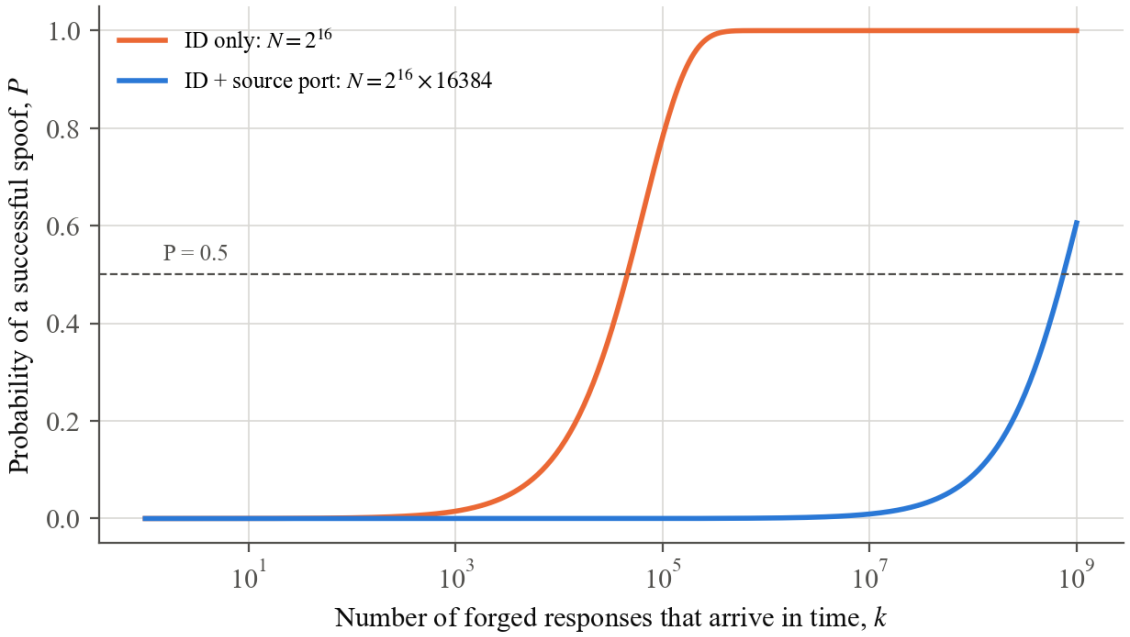


Figure 16: Probability of a successful blind spoof versus forged packets

Table 8: Fuzzing results for ParseMessage

Experiment	Inputs	Outcome
Random and mutated packets (fixed seed)	1,000,000 in 0.42 s	862,312 rejected with an error, 137,688 parsed, 0 panics
Go coverage-guided fuzzing [13], 60 s, 20 workers	33,664,027	168 new interesting inputs, no failure
Benchmarks (i7-13650HX)	ParseMessage / BuildQuery	349.5 ns / 125.8 ns per operation

No input caused a panic or hang, and parsing costs about 0.35 µs per reply, so resolution time is dominated entirely by the network.

5.6 Findings of the Security Review

A review of the source code found the following weaknesses. They do not break the implemented defences, but they matter when DNSR runs as a long-lived server and they define the remaining 20% of the implementation.

Table 9: Security review findings

No.	Finding	Recommended fix
1	No DNSSEC: on-path forgery remains possible [6]	DNSSEC validation (future work)
2	No TTL cap: a record with TTL 2^{32} s stays cached for the process lifetime	Cap TTLs (for example one day)
3	Caches never shrink; delegations never expire	Bound cache size; store NS TTLs
4	Every glueless NS name may be resolved (NXNSAttack-style amplification [17])	Limit glueless lookups per referral
5	One goroutine per packet, no rate limit; open resolver if bound publicly	Worker pool, rate limit, access list
6	TCP replies matched only by ID	Also match the question
7	No 0x20 case randomisation [18]	Randomise and verify letter case
8	Raw names printed in the trace (terminal escapes)	Escape non-printable bytes
9	Default port 5353 clashes with mDNS on Windows	Change default to 8053

5.7 Analysis

The results confirm that the implemented 80% meets the functional and security objectives. DNSR resolves real names correctly from the root, including CNAME chains across zones, glueless delegations across TLDs and truncated replies. The defences against Kaminsky-style poisoning work against hostile servers in the offline hierarchy and do not reject any legitimate delegation in the live runs. Caching delivers the expected benefit, especially the delegation cache, in line with the findings of Jung et al. [12]. The parser withstood more than 34 million fuzzing inputs. The evaluation also shows where the remaining work should go: bounded and expiring caches, limits on referral fan-out and server resources, and caching of CNAME and negative answers, followed by RTT-based server selection to cut the latency tail and DNSSEC as a longer-term extension.

6. References

[1] D. Kaminsky, "Black Ops 2008: It's the End of the Cache as We Know It," presented at Black Hat USA, Las Vegas, NV, USA, Aug. 2008.

[2] P. Mockapetris, "Domain Names – Implementation and Specification," RFC 1035, IETF, Nov. 1987.

[3] J. Damas, M. Graff, and P. Vixie, "Extension Mechanisms for DNS (EDNS(0))," RFC 6891, IETF, Apr. 2013.

[4] DNS Flag Day 2020. [Online]. Available: https://dnsflagday.net/2020/

[5] J. Dickinson, S. Dickinson, R. Bellis, A. Mankin, and D. Wessels, "DNS Transport over TCP – Implementation Requirements," RFC 7766, IETF, Mar. 2016.

[6] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS Security Introduction and Requirements," RFC 4033, IETF, Mar. 2005.

[7] A. Hubert and R. van Mook, "Measures for Making DNS More Resilient against Forged Answers," RFC 5452, IETF, Jan. 2009.

[8] R. Elz and R. Bush, "Clarifications to the DNS Specification," RFC 2181, IETF, Jul. 1997.

[9] S. Son and V. Shmatikov, "The Hitchhiker's Guide to DNS Cache Poisoning," in Proc. SecureComm 2010, LNICST, vol. 50, Springer, 2010, pp. 466–483.

[10] P. Mockapetris, "Domain Names – Concepts and Facilities," RFC 1034, IETF, Nov. 1987.

[11] M. Andrews, "Negative Caching of DNS Queries (DNS NCACHE)," RFC 2308, IETF, Mar. 1998.

[12] J. Jung, E. Sit, H. Balakrishnan, and R. Morris, "DNS Performance and the Effectiveness of Caching," IEEE/ACM Trans. Networking, vol. 10, no. 5, pp. 589–603, Oct. 2002.

[13] The Go Authors, "Go Fuzzing." [Online]. Available: https://go.dev/doc/security/fuzz/

[14] A. A. A. Donovan and B. W. Kernighan, The Go Programming Language. Boston, MA, USA: Addison-Wesley, 2015.

[15] B. Halley et al., "dnspython – A DNS Toolkit for Python." [Online]. Available: https://www.dnspython.org/

[16] Microsoft, "The default dynamic port range for TCP/IP has changed since Windows Vista and in Windows Server 2008," Microsoft Learn, KB929851. [Online]. Available: https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/default-dynamic-port-range-tcpip-chang

[17] Y. Afek, A. Bremler-Barr, and L. Shafir, "NXNSAttack: Recursive DNS Inefficiencies and Vulnerabilities," in Proc. 29th USENIX Security Symposium, 2020, pp. 631–648.

[18] D. Dagon, M. Antonakakis, P. Vixie, T. Jinmei, and W. Lee, "Increased DNS Forgery Resistance Through 0x20-Bit Encoding," in Proc. ACM CCS '08, 2008, pp. 211–222.